Minha Jornada em Estruturas de Dados com C++

Jonathan Tenório de Lima

23 de março de 2026

Sumário

1 Ordenação **2**
1.1 Insertion sort 2

2 Vetores **3**
2.1 Problema: Percorrer vetor ao contrário 3
2.1.1 Pseudocódigo 3
2.1.2 Código C++ 3
2.1.3 Observações 4

1 Ordenação

1.1 Insertion sort

Esse algoritmo de ordenação funciona semelhante a organizar as cartas do baralho na mão, durante uma partida. Inicialmente, as cartas podem estar desordenadas, então, você seleciona a segunda e compara com aquelas à esquerda, movendo a carta até a posição em esta seja a menor da sequência. A cada intervalo de interação, a carta de comparação pode ser chamada de ***carta*** e o vetor contendo as cartas será denominado $\boldsymbol{v}$. A figura 1 exemplifica o funcionamento do método de ordenação. A cada tempo, a *carta* selecionada é deslocada até a posição correspondente ordenada, o que "empurra"as outras cartas para uma posição mais adiante.

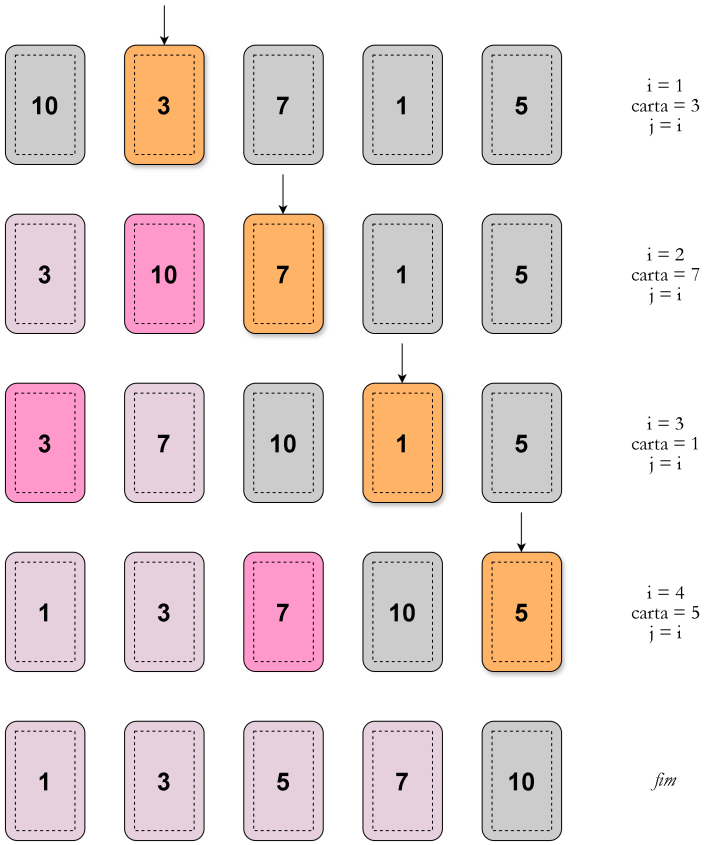

Figura 1: Exemplo do funcionamento do Insertion Sort

O pseudocódigo deste algoritmo pode ser dado como:

Algorithm 1 Insertion Sort

```
procedure InsertionSort(v)
    n ← tamanho(v)
    for i ← 1 to n − 1 do
        carta ← v[i]
        j ← i
        while j > 0 and v[j − 1] > carta do
            v[j] ← v[j − 1]
            j ← j − 1
        end while
        v[j] ← carta
    end for
end procedure
```

Também é possível escrever o código correspondente em C++ como a seguir:

```cpp

void insertionSort( std::vector<int>& v ) {
    for ( int i = 1; i < n; ++i ){
        auto carta = v[i];
        auto j = i;
        while ( j > 0 and v[j-1] > carta ) {
            v[j] = v[j-1];
            j = j - 1;
        }
        v[j] = carta;
    }
}
```

2 Vetores

2.1 Problema: Percorrer vetor ao contrário

Descrição: Dado um vetor, imprimir os elementos em ordem inversa.

2.1.1 Pseudocódigo

```
Ler vetor v
for i de tamanho(v)-1 até 0 do
    Escrever v[i]
end for
```

2.1.2 Código C++

```cpp
#include <iostream>
#include <vector>

```

```
int main() {
    std::vector<int> v = {10, 20, 30, 40, 50};

    for (std::size_t i = v.size(); i-- > 0;) {
        std::cout << v[i] << " ";
    }

    return 0;
}
```

2.1.3 Observações

Cuidado com underflow ao usar $size_t$ em loops decrescentes.